

前言

安装quartus这里就不用再次提及了，我们直接从代码入手，如果对FPGA还是不太熟悉的同学，可以看一下我之前出的Verilog速成，掌握了那个之后可以去 [HDLBits](#) 刷一下题，巩固自己的基础知识。

☺ 分频器书写(frequency_divider.v)

1. 模块接口设计

首先设计模块的输入输出接口，明确分频器需要的输入信号和输出信号。

🔗 模块接口定义：

```
1 | module frequency_divider (  
2 |     input wire      clk,      // 输入时钟  
3 |     input wire      rst_n,    // 低电平复位  
4 |     input wire [31:0] clkbase, // 基准时钟频率 (单位: Hz), 例如50MHz  
5 |     input wire [31:0] clkdiv,  // 期望输出频率 (单位: Hz), 如1000代表1kHz  
6 |     output reg       clkout    // 输出时钟  
7 | );
```

Fence 1

设计细节：

1. **输入时钟 (clk)**：模块的时钟驱动信号，所有逻辑都以此为基准。
2. **复位信号 (rst_n)**：低电平有效，用于对模块进行复位操作。
3. **基准时钟频率 (clkbase)**：指定输入时钟的频率，用于计算分频参数。
4. **期望输出频率 (clkdiv)**：用户设置的目标频率，影响分频系数。
5. **输出时钟 (clkout)**：最终生成的分频信号，供外部电路使用。

2. 计数限制值的计算

分频器的核心是通过一个计数器实现输入时钟到目标频率的分频。计数器的计数限制值 `limit` 是关键，它决定了输出时钟的翻转周期。

代码实现：

```

1 | always @* begin
2 |     if (clkdiv > 0) begin
3 |         limit = clkbase / (2 * clkdiv); // 计算分频周期
4 |     end else begin
5 |         limit = 0; // 防止除以零
6 |     end
7 | end

```

Fence 2

设计思路：

1. 计算公式

- 分频器将输入时钟频率分成目标频率，因此需要翻转输出时钟信号的周期为： $limit = \frac{clkbase}{2 \times clkdiv}$
- 这里除以 2 是因为每次翻转输出信号需要两个计数周期。

2. 除零保护

- 当 `clkdiv` 为 0 时，可能发生除零错误，因此在这种情况下直接将 `limit` 设置为 0，以避免逻辑错误。

3. 时钟翻转逻辑

利用一个计数器完成对输入时钟周期的计数，当计数值达到限制值时，翻转输出时钟信号并将计数器清零。

代码实现：

```

1 | always @(posedge clk or negedge rst_n) begin
2 |     if (!rst_n) begin
3 |         counter <= 0; // 复位计数器
4 |         clkout <= 0; // 复位输出时钟
5 |     end else begin
6 |         if (limit > 0) begin
7 |             if (counter >= limit - 1) begin
8 |                 clkout <= ~clkout; // 翻转输出时钟
9 |                 counter <= 0; // 计数器归零
10 |             end else begin
11 |                 counter <= counter + 1; // 计数器加一
12 |             end
13 |         end
14 |     end
15 | end

```

Fence 3

设计细节：

1. 复位逻辑

- 当复位信号有效时，将计数器和输出时钟清零，确保模块处于初始状态。

2. 计数逻辑

- 每个输入时钟周期计数器加一，直到达到限制值。
- 达到限制值时翻转输出信号，完成一次完整的时钟周期。

3. 时钟翻转

- 利用逻辑运算符 $\sim$ 翻转输出信号，实现时钟波形的高低电平切换。

4. 应用示例

假设输入时钟频率为 50MHz，目标输出频率为 1kHz：

1. 参数设置

- `clkbase = 50_000_000` (50MHz)

- `clkdiv = 1_000` (1kHz)

2. 限制值计算： $limit = \frac{50000000}{2 \times 1000} = 25000$ 每 25,000 个输入时钟周期翻转一次输出信号。

3. 结果：输出信号

5. 整体代码

```
1  module frequency_divider (  
2      input wire      clk,        // 输入时钟  
3      input wire      rst_n,      // 低电平复位  
4      input wire [31:0] clkbase,   // 基准时钟频率 (单位: Hz), 例如50MHz  
5      input wire [31:0] clkdiv,    // 期望输出频率 (单位: Hz), 如1000代表1kHz  
6      output reg       clkout      // 输出时钟  
7  );  
8      reg [31:0] counter; // 计数器  
9      reg [31:0] limit;   // 计数限制值  
10  
11     // 计算每个分频周期的计数限制  
12     always @* begin  
13         if (clkdiv > 0) begin  
14             limit = clkbase / (2 * clkdiv); // 计算分频周期  
15         end else begin  
16             limit = 0; // 防止除以零  
17         end  
18     end  
19  
20     // 计数逻辑  
21     always @(posedge clk or negedge rst_n) begin  
22         if (!rst_n) begin
```

```
23         counter <= 0; // 复位计数器
24         clkout  <= 0; // 复位输出时钟
25     end else begin
26         if (limit > 0) begin
27             if (counter >= limit - 1) begin
28                 clkout <= ~clkout; // 翻转输出时钟
29                 counter <= 0; // 计数器归零
30             end else begin
31                 counter <= counter + 1; // 计数器加一
32             end
33         end
34     end
35 end
36 endmodule
37
```

Fence 4

☹ 数码管模块

在数码管显示中，我们的核心任务是控制每一个数码管正确显示数字和符号。这里的实现逻辑分为三部分：

- 1. **段选与位选的基础知识**：控制每个数码管的亮灭规则及数字编码。
- 2. **底层逻辑实现**：实现段码和位码的生成。
- 3. **应用层逻辑**：通过动态扫描实现多位数码管的显示。

段码位码知识

🔗 1. 什么是段选和位选？

数码管的显示由 **段选（Segment Selection）** 和 **位选（Position Selection）** 两部分组成：

- **段选（SEG_A ~ SEG_DP）**：控制某一数码管中每一段的亮灭状态。
每一段可以通过高电平或低电平来点亮。例如，显示数字“2”时，点亮的段为 a, b, g, e, d，而其余的段熄灭。
- **位选（SEG_COM1 ~ SEG_COM8）**：控制当前显示的是哪一颗数码管。
这里使用了 **共阳极数码管**，所以低电平有效，即 0 代表选中，1 代表关闭。

为了更直观，我们可以将段选和位选的关系看成一个舞台灯光秀：

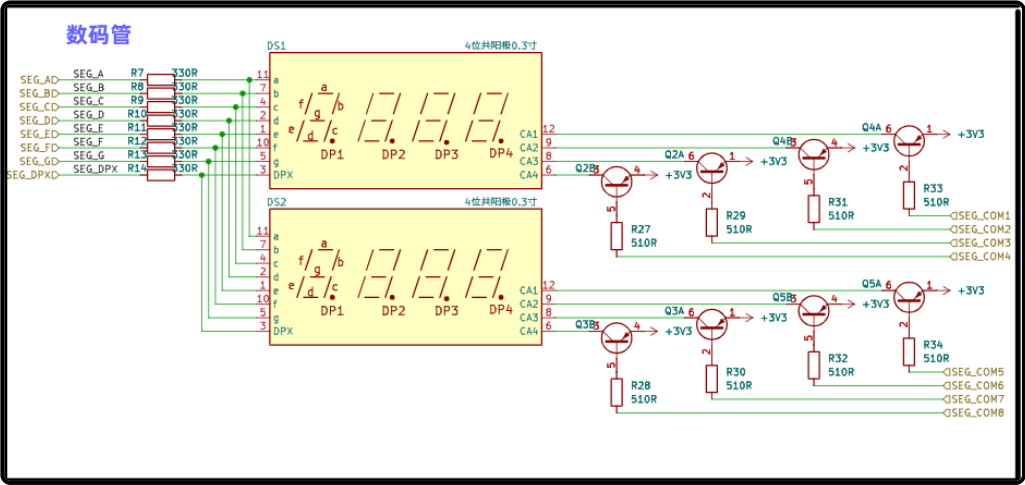


Figure 1

- **段选** 就像控制每个灯泡亮灭的开关，决定一个数码管显示的内容。
- **位选** 就像决定哪个灯被聚光灯照亮，选择当前显示的数码管。

2. 段码的定义

在共阳极数码管中，**段选的低电平代表点亮，高电平代表熄灭**。我们可以为每个数字定义一个固定的段码表：

```

1 // 段码表：每个数字对应的七段显示编码
2 localparam [7:0] SEG_0 = 8'b1100_0000; // 数字 0
3 localparam [7:0] SEG_1 = 8'b1111_1001; // 数字 1
4 localparam [7:0] SEG_2 = 8'b1010_0100; // 数字 2
5 localparam [7:0] SEG_3 = 8'b1011_0000; // 数字 3
6 localparam [7:0] SEG_4 = 8'b1001_1001; // 数字 4
7 localparam [7:0] SEG_5 = 8'b1001_0010; // 数字 5
8 localparam [7:0] SEG_6 = 8'b1000_0010; // 数字 6
9 localparam [7:0] SEG_7 = 8'b1111_1000; // 数字 7
10 localparam [7:0] SEG_8 = 8'b1000_0000; // 数字 8
11 localparam [7:0] SEG_9 = 8'b1001_0000; // 数字 9
12 localparam [7:0] SEG_NULL = 8'b1111_1111; // 熄灭

```

Fence 5

例如，数字“2”的段码为 `1010_0100`，它表示点亮 `a`，`b`，`g`，`e`，`d` 段。这种编码可以看成数码管显示的“字典”，后续逻辑会反复引用它。

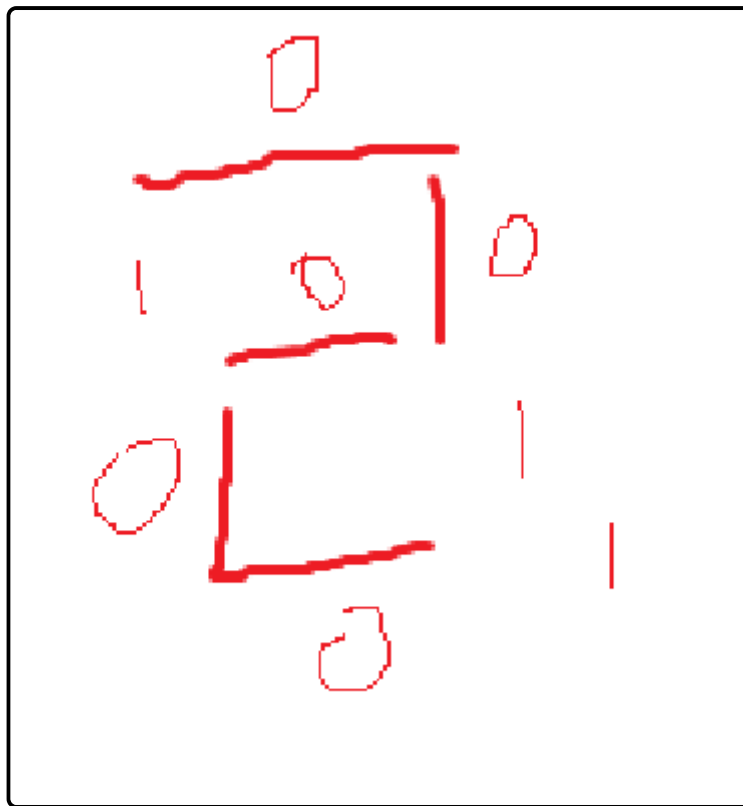


Figure 2

底层逻辑实现

底层逻辑主要解决“**如何为单个数码管生成段码和位码**”的问题。

模块 `seg_driver` 会根据输入的数字和小数点状态，自动生成对应的段码和位选信号。

模块设计

模块的接口如下：

```
1 module seg_driver (
2     input      [3:0] digit,          // 输入的数字 (0-9)
3     input      rst_n,                // 复位信号 (低电平有效)
4     input      current_dp,          // 小数点使能信号
5     input      [2:0] position,       // 当前显示的位 (0~7)
6     output reg [7:0] digit_sel,      // 位选信号
7     output reg [7:0] segment_data    // 段选信号
8 );
```

Fence 6

实现步骤

1. 段码生成

段码的生成是根据输入的数字 `digit` 来查询段码表，并通过小数点状态 `current_dp` 控制最高位：

- 如果 `current_dp` 为 1，小数点点亮（最高位为 0）。
- 如果 `current_dp` 为 0，小数点熄灭。

代码实现如下：

```
1 always @(*) begin
2     if (!rst_n) begin
3         segment_data <= 8'b1111_1111; // 复位时熄灭所有段码
4     end else begin
5         case (digit)
6             4'd0: segment_data = SEG_0;
7             4'd1: segment_data = SEG_1;
8             4'd2: segment_data = SEG_2;
9             4'd3: segment_data = SEG_3;
10            4'd4: segment_data = SEG_4;
11            4'd5: segment_data = SEG_5;
12            4'd6: segment_data = SEG_6;
13            4'd7: segment_data = SEG_7;
14            4'd8: segment_data = SEG_8;
15            4'd9: segment_data = SEG_9;
16            default: segment_data = 8'b1111_1111; // 无效输入时熄灭所有段码
17        endcase
18        if (current_dp) segment_data[7] = 1'b0; // 点亮小数点
19    end
20 end
```

Fence 7

2. 位选生成

位选的生成是通过当前位置信号 `position` 计算的。因为低电平有效，所以我们使用取反操作：

```
1 | digit_sel = ~(8'b0000_0001 << position); // 激活对应的位
```

Fence 8

通过这种设计，段码和位码可以独立生成，后续的应用层逻辑会结合它们实现多位数码管的动态显示。

完整代码

```
1 module seg_driver (
2     input    [3:0] digit,        // 输入的十进制数字 (0-9)
3     input    rst_n,             // 复位信号 (低电平有效)
4     input    current_dp,        // 小数点使能信号
5     input    [2:0] position,    // 当前显示的位
6     output reg [7:0] digit_sel, // 位选信号
7     output reg [7:0] segment_data // 段选信号
8 );
9
10 // 段码定义 (每个数字对应的七段显示编码)
11 localparam [7:0] SEG_0 = 8'b1100_0000; //0
12 localparam [7:0] SEG_1 = 8'b1111_1001; //1
13 localparam [7:0] SEG_2 = 8'b1010_0100; //2
14 localparam [7:0] SEG_3 = 8'b1011_0000; //3
15 localparam [7:0] SEG_4 = 8'b1001_1001; //4
16 localparam [7:0] SEG_5 = 8'b1001_0010; //5
17 localparam [7:0] SEG_6 = 8'b1000_0010; //6
18 localparam [7:0] SEG_7 = 8'b1111_1000; //7
19 localparam [7:0] SEG_8 = 8'b1000_0000; //8
20 localparam [7:0] SEG_9 = 8'b1001_0000; //9
21 localparam [7:0] SEG_NULL = 8'b1111_1111; //灭
22
23 // 根据输入的数字和小数点状态生成段码
24 always @(*) begin
25     if (!rst_n) begin
26         segment_data <= 8'b1111_1111; // 复位时熄灭所有段码
27         digit_sel <= 8'b1111_1111; // 复位时关闭所有位选
28     end else begin
29         // 根据输入的数字选择对应的段码
30         case (digit)
31             4'd0: segment_data = SEG_0;
32             4'd1: segment_data = SEG_1;
33             4'd2: segment_data = SEG_2;
34             4'd3: segment_data = SEG_3;
35             4'd4: segment_data = SEG_4;
36             4'd5: segment_data = SEG_5;
37             4'd6: segment_data = SEG_6;
38             4'd7: segment_data = SEG_7;
39             4'd8: segment_data = SEG_8;
40             4'd9: segment_data = SEG_9;
41             default: segment_data = 8'b1111_1111; // 无效输入时熄灭所有段码
42         endcase
43
44         // 如果 decimal_en 为 1, 则激活小数点 (最高位为小数点位)
45         if (current_dp) segment_data[7] = 1'b0;
46
47         // 根据当前位置 `position` 控制位选信号
```

```
48         digit_sel = ~(8'b0000_0001 << position); // 按位激活
49     end
50 end
51
52 endmodule
53
```

Fence 9

应用层逻辑

应用层逻辑解决“如何让多位数码管动态显示不同内容”的问题。它通过“扫描显示”的方式，在每一时刻点亮一颗数码管，并快速切换，达到“多位同时显示”的效果。

模块 `seg_proc` 整合了分频器和驱动模块，实现了多位动态扫描。

📁 模块设计

模块接口如下：

```
1 module seg_proc (
2     input clk,    // 50MHz 时钟信号
3     input rst_n,  // 低电平复位信号
4     input [31:0] seg_number_in, // 输入的8组数字（每组4位BCD码）
5     input [7:0] dp, // 小数点状态（每个位对应1个）
6     output [7:0] dula, // 段选信号
7     output [7:0] wela // 位选信号
8 );
```

Fence 10

📁 实现步骤

1. 分频器

通过分频器将50MHz时钟分频为1kHz，用于动态扫描的节奏：

```
1 frequency_divider u_frequency_divider (
2     .clk(clk),
3     .rst_n(rst_n),
4     .clkbase(50_000_000),
5     .clkdiv(1_000),
6     .clkout(clk_1k)
7 );
```

Fence 11

2. 动态扫描逻辑

通过 `bits` 记录当前显示的位（0~7），每1ms切换到下一位：

```
1 | always @(posedge clk_1k or negedge rst_n) begin
2 |     if (!rst_n)
3 |         bits <= 3'd0; // 复位时从第0位开始
4 |     else
5 |         bits <= (bits == 3'd7) ? 3'd0 : bits + 3'd1; // 循环切换
6 | end
```

Fence 12

然后，根据当前的 `bits`，提取对应的数字内容和小数点状态：

```
1 | always @(*) begin
2 |     current_num = seg_number_in[(7-bits)*4+:4]; // 提取数字
3 |     current_dp  = dp[7-bits]; // 提取小数点状态
4 | end
```

Fence 13

这一段代码的核心是提取当前需要显示的数字数据，`current_num` 负责存储当前位的 4 位 BCD 码（0~9）。我们分解解释：

表达式解析

```
1 | current_num = seg_number_in[(7-bits)*4+:4];
```

Fence 14

1. `seg_number_in`

这是一个 32 位的输入信号，表示 8 个数码管需要显示的内容。每 4 位对应一个数码管上的数字（BCD 码）。

例如：

```
1 | seg_number_in = {4'b0000, 4'b0001, 4'b0010, 4'b0011, 4'b0100, 4'b0101, 4'b0110,
4'b0111};
```

Fence 15

表示从第 1 到第 8 位数码管分别显示 0, 1, 2, 3, 4, 5, 6, 7。

2. `(7-bits)`

`bits` 是当前的位索引信号，取值范围是 0~7，表示当前正在扫描的数码管位置：

○ `bits = 0` 表示第 8 位数码管（最高位）。

○ `bits = 7` 表示第 1 位数码管（最低位）。

因为输入数据从最高位（第 8 位）开始排序，而扫描是从最低位开始的，所以用 `(7-bits)` 将索引反转。

3. $(7-\text{bits}) \times 4$

每 4 位对应一个数码管显示的数据， $(7-\text{bits}) \times 4$ 计算当前位在 `seg_number_in` 中的起始位置。例如：

- 当 `bits=0` 时， $(7-0) \times 4=28$ ，对应第 8 位数码管的起始位置；
- 当 `bits=7` 时， $(7-7) \times 4=0$ ，对应第 1 位数码管的起始位置。

4. `+:4`

这里是 Verilog 的切片操作，表示从起始位置 $(7-\text{bits}) \times 4$ 开始，提取连续 4 位。例如：

- 当 `bits=0` 时，提取 `seg_number_in[28:25]`；
- 当 `bits=7` 时，提取 `seg_number_in[3:0]`。

总结

这段代码的作用是从 `seg_number_in` 中，根据当前扫描位 `bits`，提取对应的 4 位 BCD 码，作为当前数码管要显示的数字。

实例举例

假设 `seg_number_in = 32'b0000_0001_0010_0011_0100_0101_0110_0111`，对应数码管显示 0, 1, 2, 3, 4, 5, 6, 7。

- 当 `bits=0`，对应第 8 位：`current_num = seg_number_in[28+:4] = 4'b0000`，显示 0。
- 当 `bits=7`，对应第 1 位：`current_num = seg_number_in[0+:4] = 4'b0111`，显示 7。

3. 驱动模块例化

将提取的数字和状态送入驱动模块生成段码和位码：

```

1 | seg_driver u_seg_driver (
2 |     .digit(current_num),
3 |     .rst_n(rst_n),
4 |     .current_dp(current_dp),
5 |     .position(bits),
6 |     .digit_sel(wela),
7 |     .segment_data(dula)
8 | );

```

Fence 16

完整代码

```

1 | module seg_proc (
2 |     input clk,
3 |     input rst_n,
4 |     input [31:0] seg_number_in,
5 |     input [7:0] dp,
6 |     output [7:0] dula,
7 |     output [7:0] wela
8 | );
9 |     wire clk_1k;

```

```
10     reg [2:0] bits;
11     reg [3:0] current_num;
12     reg      current_dp;
13
14     frequency_divider u_frequency_divider (
15         .clk(clk),
16         .rst_n(rst_n),
17         .clkbase(50_000_000),
18         .clkdiv(1_000),
19         .clkout(clk_1k)
20     );
21
22     always @(*) begin
23         current_num = seg_number_in[(7-bits)*4+:4];
24         current_dp  = dp[7-bits];
25     end
26
27     seg_driver u_seg_driver (
28         .digit(current_num),
29         .rst_n(rst_n),
30         .current_dp(current_dp),
31         .position(bits),
32         .digit_sel(wela),
33         .segment_data(dula)
34     );
35
36     always @(posedge clk_1k or negedge rst_n) begin
37         if (!rst_n)
38             bits <= 3'd0;
39         else
40             bits <= (bits == 3'd7) ? 3'd0 : bits + 3'd1;
41     end
42 endmodule
```

Fence 17

☹ 按键模块

按键知识

我们可以从原理图中看到，按键的右边是地，左边是一个3.3V的电压和我们的S1 ~ S4的IO口，那么这个电路的意思就是，当我按下按键的时候，对应的S口是低电平（因为和GND导通）

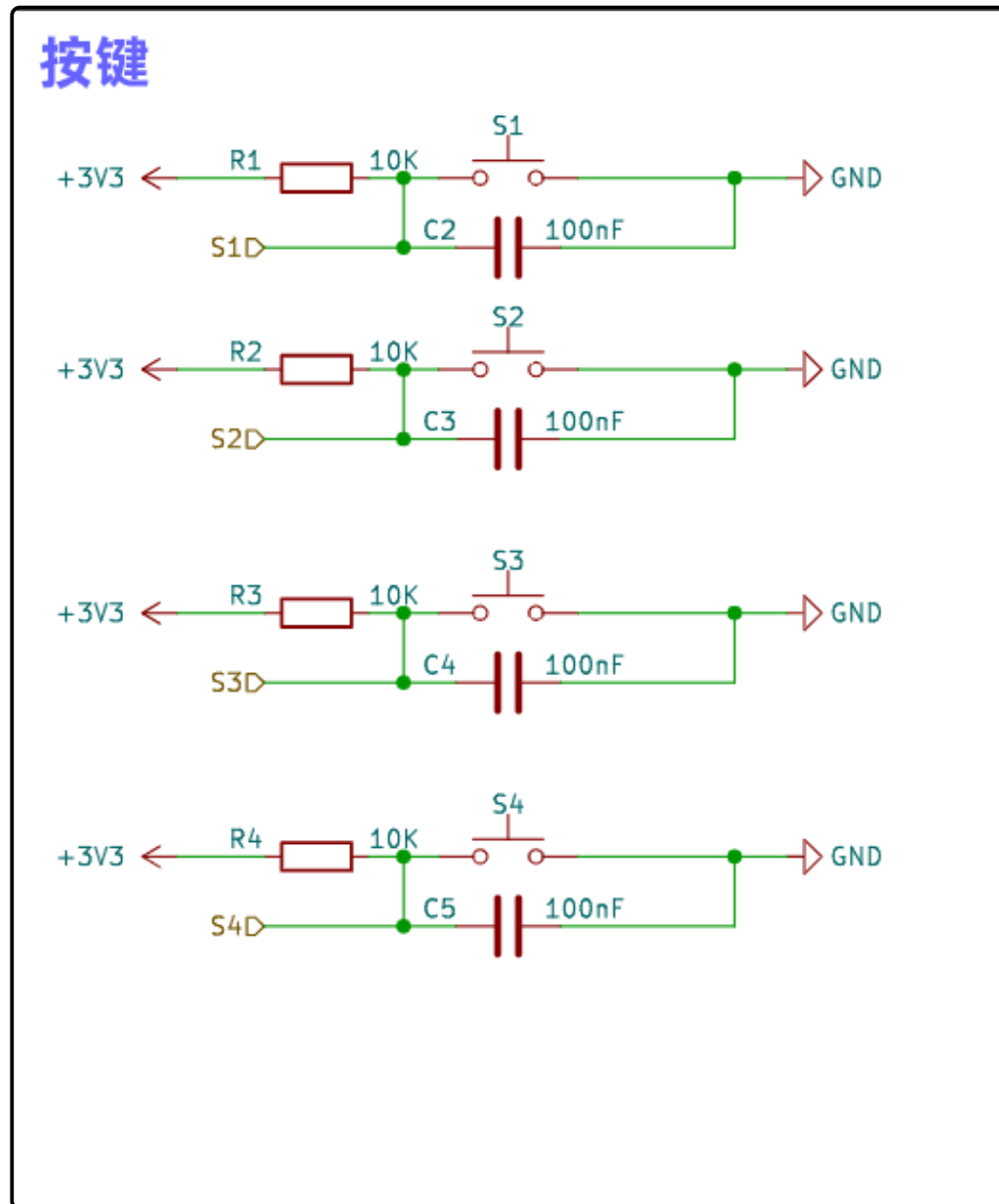


Figure 3

因为众所周知的问题，按键是需要进行消抖处理的，所有我们这里也有一个消抖的思想（并非状态机），如果之前看过单片机的课程的话，会对这个三行代码消抖印象比较深刻

这个在我以前的视频 [蓝桥杯单片机按键考点](#) 里面提到过，点击这行字体就可以跳转过去，我这里简单说明一下，我们的逻辑如下，Key_Val, Key_Old, Key_Down, Key_Up, 这几个变量都是下面的逻辑，我们这里对其的原理进行解析

瞬时值获取-> 获取按下的值，获取抬起的值->更新一下值

Down和Up都是瞬时值，Old比Val延后10ms，基于这个特性我们可以进行书写代码(注意，这里是时序里面，100Hz也就是10ms触发一次扫描)

```

1 // 按键状态检测逻辑
2 always @(posedge clk_100 or negedge rst_n) begin
3     if (!rst_n) begin
4         // 复位时将所有状态设为未按下
5         val <= 4'b1111; // 假设按键为低电平有效，初始化为高电平未按下状态
6         old <= 4'b1111; // 上一个状态初始化为高电平
7         down <= 4'b0000; // 初始时没有按键按下
8         up <= 4'b0000; // 初始时没有按键释放
9     end
10    else begin
11        old <= val; // 保存当前状态到 old 以便下一个周期比较
12        val <= key_in; // 更新 val 为当前的按键状态
13
14        // 检测按键按下: val 和 old 比较，下降沿检测
15        down <= val & (val ^ old); // val 为低且 val 与 old 不同，说明按键刚被按下
16
17        // 检测按键释放: val 和 old 比较，上升沿检测
18        up <= ~val & (val ^ old); // val 为高且 val 与 old 不同，说明按键刚被释放
19    end
20 end

```

Fence 18

如果理解不了代码和这个意思，我就拿一个 🍷 来进行验证，比如我们按下/抬起S1来做一下验证

键盘状态	Key_Old	Key_Val	Key_Old^Key_Val	Key_Down	Key_Up
未按下	0000	0000	0000^0000=0000	0000	0000
按下过程 中 (10ms)	0000	1000	0000^1000=1000	1000&1000=1000	0111&1000=0000
按下稳定 (10ms 后)	1000	1000	1000^1000=0000	1000&0000=0000	0111&0000=0000
抬起过程 (10ms)	1000	0000	0100^0000=0100	0000&0100=0000	1111&1000=1000

Table 1

我们可以从这里看出，当我们按下的时候，那一瞬间Down会是1000（注意是一瞬间），其余时候都是0；同理Up也是一样，当我们抬起的瞬间Up会是1000（注意是一瞬间），其余时候是0

底层逻辑实现

🔗 模块设计

其实我们底层就是把四个按键的当前状态传入然后进行一个up和down的输出而已，这里主要是要注意，按键的传入和up与down输出都是四位的就行了

```

1  module key_ctrl (
2      input          clk_100, // 输入时钟信号(100Hz)
3      input          rst_n,   // 低电平复位信号
4      input  [3:0] key_in,    // 4 个按键输入信号
5      output reg [3:0] down,  // 按键按下检测信号（下降沿触发）
6      output reg [3:0] up    // 按键释放检测信号（上升沿触发）
7  );
8

```

Fence 19

完整代码

直接把我们的上面提到的消抖逻辑放进去就可以了（注意一下内部的那两个变量是reg）

```

1  module key_ctrl (
2      input          clk_100, // 输入时钟信号(100Hz)
3      input          rst_n,   // 低电平复位信号
4      input  [3:0] key_in,    // 4 个按键输入信号
5      output reg [3:0] down,  // 按键按下检测信号（下降沿触发）
6      output reg [3:0] up    // 按键释放检测信号（上升沿触发）
7  );
8
9      reg [3:0] val; // 当前按键状态
10     reg [3:0] old; // 上一个时钟周期的按键状态
11
12     // 按键状态检测逻辑
13     always @(posedge clk_100 or negedge rst_n) begin
14         if (!rst_n) begin
15             // 复位时将所有状态设为未按下
16             val <= 4'b1111; // 假设按键为低电平有效，初始化为高电平未按下状态
17             old <= 4'b1111; // 上一个状态初始化为高电平
18             down <= 4'b0000; // 初始时没有按键按下
19             up <= 4'b0000; // 初始时没有按键释放
20         end
21         else begin
22             old <= val; // 保存当前状态到 old 以便下一个周期比较
23             val <= key_in; // 更新 val 为当前的按键状态
24
25             // 检测按键按下: val 和 old 比较, 下降沿检测
26             down <= val & (val ^ old); // val 为低且 val 与 old 不同, 说明按键刚被
按下
27
28             // 检测按键释放: val 和 old 比较, 上升沿检测
29             up <= ~val & (val ^ old); // val 为高且 val 与 old 不同, 说明按键刚被
释放
30         end
31     end
32 endmodule
33

```

Fence 20

应用层逻辑

这里其实没什么太多需要说明的，因为需要结合具体问题具体分析，我这里写的示例是按键控制led流水灯和数码管显示固定的数据，所以这里就会有一个模式的输出（通过按键修改LED的模式）

模块设计

我们还是老样子，需要输入系统的时钟和一个复位信号进去，然后把四个按键当前的状态进行输入，下面的输出在实际使用中可以自行替换，我这里是控制LED模式，所以是这样声明的，本质上你随便怎么声明输出的是啥都可以

```
1 module key_proc (
2     input        clk,        // 系统时钟信号
3     input        rst_n,      // 低电平复位信号
4     input [3:0]  key_in,     // 4 个按键输入信号
5     output reg [1:0] mode    // 模式输出，用于切换 LED 显示模式
6 );
```

Fence 21

实现步骤

1. 分频器和消抖例化

我们在这里首先例化了分频器和消抖模块，将分频器分频后的时钟信号直接送入上面构建好的消抖模块进行消抖

```
1 wire [3:0] down; // 每个按键的按下检测信号
2 wire [3:0] up;  // 每个按键的松开检测信号
3 wire      clk_100; //100Hz
4 frequency_divider u_frequency_divider (
5     .clk      (clk),
6     .rst_n    (rst_n),
7     .clkbase(50_000_000),
8     .clkdiv  (100),
9     .clkout   (clk_100)
10 );
11
12 // 实例化按键消抖模块 key_ctrl, 用于按键消抖处理
13 key_ctrl key_ctrl_inst (
14     .clk      (clk_100), // 系统时钟输入
15     .rst_n    (rst_n),   // 低电平复位信号
16     .key_in   (key_in),  // 按键输入信号
17     .down     (down),    // 按下检测输出
18     .up       (up),      // 松开检测输出
19 );
20
```

Fence 22

2. 按键实例使用

这里就是根据题目的要求来的使用了，我这里是写的一个流水灯的模式切换，按下哪个按钮就切换到哪个模式，这里是可以直接自定义的

```

1 // 根据按键按下状态控制模式切换
2     always @(posedge clk_100 or negedge rst_n) begin
3         if (!rst_n) begin
4             mode <= 2'b00; // 复位时模式设为初始模式 0
5         end
6         else begin
7             // 按键 0-3 分别控制模式 0-3
8             if (down[0])
9                 mode <= 2'b00; // 按键0: 切换到模式 0
10            else if (down[1])
11                mode <= 2'b01; // 按键1: 切换到模式 1
12            else if (down[2])
13                mode <= 2'b10; // 按键2: 切换到模式 2
14            else if (down[3])
15                mode <= 2'b11; // 按键3: 切换到模式 3
16        end
17    end

```

Fence 23

完整代码

```

1 module key_proc (
2     input          clk,        // 系统时钟信号
3     input          rst_n,      // 低电平复位信号
4     input [3:0]    key_in,     // 4 个按键输入信号
5     output reg [1:0] mode      // 模式输出，用于切换 LED 显示模式
6 );
7
8 wire [3:0] down; // 每个按键的按下检测信号
9 wire [3:0] up;   // 每个按键的松开检测信号
10 wire      clk_100; //100Hz
11 frequency_divider u_frequency_divider (
12     .clk      (clk),
13     .rst_n    (rst_n),
14     .clkbases(50_000_000),
15     .clkdiv   (100),
16     .clkout   (clk_100)
17 );
18
19 // 实例化按键消抖模块 key_ctrl1, 用于按键消抖处理
20 key_ctrl1 key_ctrl1_inst (
21     .clk      (clk_100), // 系统时钟输入
22     .rst_n    (rst_n),   // 低电平复位信号
23     .key_in   (key_in),  // 按键输入信号
24     .down     (down),    // 按下检测输出
25     .up       (up),      // 松开检测输出
26 );
27
28 // 根据按键按下状态控制模式切换
29 always @(posedge clk_100 or negedge rst_n) begin
30     if (!rst_n) begin
31         mode <= 2'b00; // 复位时模式设为初始模式 0

```

```
32         end
33     else begin
34         // 按键 0-3 分别控制模式 0-3
35         if (down[0])
36             mode <= 2'b00; // 按键0: 切换到模式 0
37         else if (down[1])
38             mode <= 2'b01; // 按键1: 切换到模式 1
39         else if (down[2])
40             mode <= 2'b10; // 按键2: 切换到模式 2
41         else if (down[3])
42             mode <= 2'b11; // 按键3: 切换到模式 3
43         end
44     end
45 endmodule
46
```

Fence 24

☹ LED模块

基础知识

从这个原理图我们就能看出来，二极管左边是我们的IO口，所以我们只需要给对应IO口低电平，就可以实现点亮LED灯

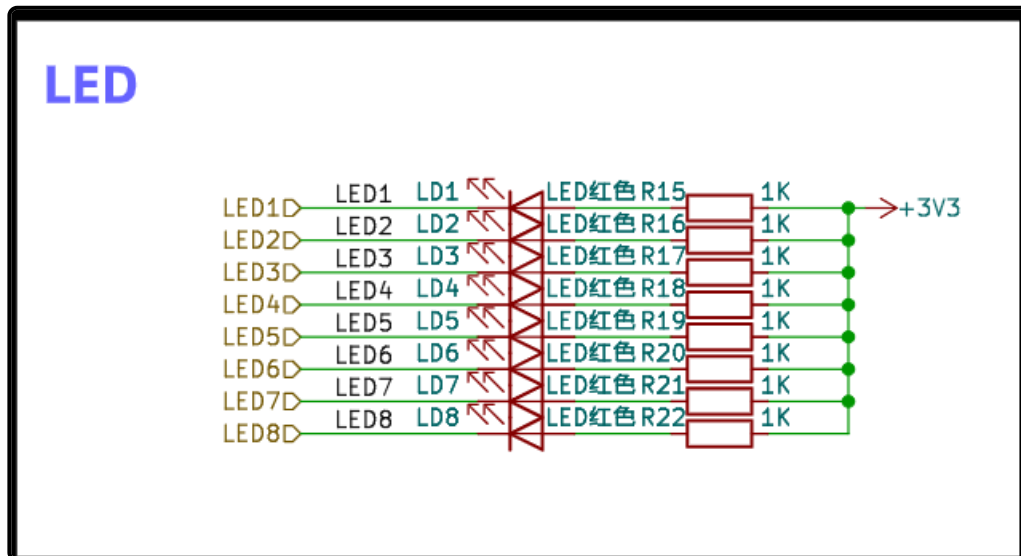


Figure 4

底层逻辑实现

其实他的底层非常好写，我们直接给IO口高低电平就行了，但是由于这个是低电平点亮，我们调用的时候会比较的麻烦，所以我们在底层这里就直接给他取反就好了这里的led就是我们最终会显示的led的口，当我们调用的时候，传入的数据是和我们直觉一致的，就比如我们让LED1点亮，其他熄灭，我们第一反应就应该是1000_0000这种，但其实是0111_1111这种，所以在底层只需要将其取反就好，我们就可以用第一种方式在应用层来进行使用

```
1 module led_display (  
2     input    [7:0] led_pattern,  
3     output reg [7:0] led  
4 );  
5     always @(*) begin  
6         led = ~led_pattern;  
7     end  
8 endmodule  
9
```

Fence 25

应用层逻辑实现

模块设计

就像我在key里面说的一样，我们这里做的是一个按键控制流水灯模式，所以有一个mode的输入，其他的输入输出都很常规，就是时钟、复位、LED输出

```

1  module led_proc (
2      input      clk,      // 50MHz 时钟输入
3      input      rst_n,    // 低电平复位信号
4      input  [1:0] mode,    // 模式选择信号
5      output [7:0] led      // LED 输出
6  );

```

Fence 26

实现步骤

1. 分频器和LED底层实例化

我们这里的流水灯是1s切换一次的，也就是说我们需要一个1Hz的信号

```

1  // 实例化 frequency_divider 模块，将 50MHz 时钟分频到 1Hz
2      frequency_divider freq_div_inst (
3          .clk      (clk),          // 原始时钟信号 50MHz
4          .rst_n    (rst_n),        // 低电平复位信号
5          .clkbases(50_000_000),    // 基准时钟频率为 50MHz
6          .clkdiv   (1),            // 期望输出频率为 1Hz
7          .clkout   (clk_1)         // 输出分频后的 1Hz 时钟信号
8      );
9  // 实例化 LED 显示模块，将 led_pattern 映射到实际 LED 输出
10     led_display led_display_inst (
11         .led_pattern(led_pattern), // 输入的 LED 控制模式
12         .led        (led)          // 映射到实际 LED 显示
13     );

```

Fence 27

2. 流水灯实现

这里和按键的实现一样，也是任意的，因为我们这里是流水灯，所以说这里就是一个流水灯的实现，比如模式0就会一直将灯往左边移动，就是我在速通那里说过的方式。

```

1  // 根据 mode 模式和 1Hz 慢时钟信号 clk_1 控制 LED 模式
2  always @(posedge clk_1 or negedge rst_n) begin
3      if (!rst_n) begin
4          led_pattern <= 8'b0000_0001; // 复位时默认模式（初始值）
5      end
6      else begin
7          case (mode)
8              2'b00:
9                  led_pattern <= {led_pattern[6:0], led_pattern[7]}; // 左移模
10             式
11              2'b01:

```

```
11         led_pattern <= {led_pattern[0], led_pattern[7:1]}; // 右移模  
式  
12         2'b10:  
13             led_pattern <= ~led_pattern; // 闪烁模式  
14         2'b11:  
15             led_pattern <= 8'b1010_1010; // 固定模式  
16         default:  
17             led_pattern <= 8'b0000_0001; // 默认模式为左移  
18     endcase  
19 end  
20 end  
21
```

Fence 28

TOP模块

这个就是我们的一个整体的模块了

模块设计

这里面包含了我们用到的这些端口，时间、复位、按键、数码管的段选位选与LED灯

```
1 module top (
2     input      clk,
3     input      rst_n,
4     input [3:0] key_in,
5     output [7:0] led,
6     output [7:0] dula,
7     output [7:0] wela,
8 );
```

Fence 29

数据定义

我们在这里用作测试显示数据为1314521，并且让第五位数码管的小数点点亮，这里注意，我们的值全部都是4'b的值，都是4位的，如果后续要使用我的数码管模块，需要将数据处理为四位，比如我这里的直接定义一个4位的变量来进行接受当前值，否则数码管显示会有问题

```
1 localparam num0 = 1314;
2 localparam num1 = 521;
3
4 wire [3:0] num0_thousand;
5 wire [3:0] num0_hundred;
6 wire [3:0] num0_ten;
7 wire [3:0] num0_one;
8
9 wire [3:0] num1_hundred;
10 wire [3:0] num1_ten;
11 wire [3:0] num1_one;
12 assign num0_thousand = num0 / 1000;
13 assign num0_hundred = (num0 % 1000) / 100;
14 assign num0_ten = (num0 % 100) / 10;
15 assign num0_one = num0 % 10;
16
17 assign num1_hundred = num1 / 100;
18 assign num1_ten = (num1 % 100) / 10;
19 assign num1_one = num1 % 10;
20
21 wire [1:0] mode;
22 wire [7:0] dp;
23
24 assign dp = 8'b0000_1000;
```

Fence 30

实例化模块

定义都好了之后，我们就把这些模块一个一个的实例化就可以了

```

1 // 实例化按键处理模块
2 key_proc key_proc_inst (
3     .clk    (clk),
4     .rst_n  (rst_n),
5     .key_in (key_in),
6     .mode   (mode)
7 );
8
9 // 实例化LED处理模块
10 led_proc led_proc_inst (
11     .clk    (clk),
12     .rst_n  (rst_n),
13     .mode   (mode),
14     .led    (led)
15 );
16
17 // 实例化数码管处理模块
18 seg_proc u_seg_proc (
19     .clk(clk),
20     .rst_n(rst_n),
21     .seg_number_in({
22         num0_thousand, num0_hundred, num0_ten,
23         num0_one, 4'd10, num1_hundred, num1_ten, num1_one
24     }),
25     .dp(dp),
26     .dula(dula),
27     .wela(wela)
28 );

```

Fence 31

完整代码

```

1 module top (
2     input      clk,
3     input      rst_n,
4     input  [3:0] key_in,
5     output [7:0] led,
6     output [7:0] dula,
7     output [7:0] wela
8 );
9     localparam num0 = 1314;
10    localparam num1 = 521;
11    wire [3:0] num0_thousand;
12    wire [3:0] num0_hundred;
13    wire [3:0] num0_ten;
14    wire [3:0] num0_one;
15
16    wire [3:0] num1_hundred;
17    wire [3:0] num1_ten;
18    wire [3:0] num1_one;
19    assign num0_thousand = num0 / 1000;

```



```

20     assign num0_hundred = (num0 % 1000) / 100;
21     assign num0_ten = (num0 % 100) / 10;
22     assign num0_one = num0 % 10;
23
24     assign num1_hundred = num1 / 100;
25     assign num1_ten = (num1 % 100) / 10;
26     assign num1_one = num1 % 10;
27
28
29     wire [1:0] mode;
30     wire [7:0] dp;
31
32     assign dp = 8'b0000_1000;
33     // 实例化按键处理模块
34     key_proc key_proc_inst (
35         .clk (clk),
36         .rst_n (rst_n),
37         .key_in(key_in),
38         .mode (mode)
39     );
40
41     // 实例化LED处理模块
42     led_proc led_proc_inst (
43         .clk (clk),
44         .rst_n(rst_n),
45         .mode (mode),
46         .led (led)
47     );
48
49     // 实例化数码管处理模块
50     seg_proc u_seg_proc (
51         .clk(clk),
52         .rst_n(rst_n),
53         .seg_number_in({
54             num0_thousand, num0_hundred, num0_ten,
55             num0_one, 4'd10, num1_hundred, num1_ten, num1_one
56         }),
57         .dp(dp),
58         .dula(dula),
59         .wela(wela)
60     );
61
62 endmodule
63

```

Fence 32



☹ 工程引脚

我们需要查看一下我们的引脚，通过查看官方给的pin可以知道，我们要用到这些

1	引脚名	FPGA绑定...	引脚功能	
2	GCLK	PIN_23	系统时钟	
3	S1	PIN_88	用户按键1	
4	S2	PIN_89	用户按键2	
5	S3	PIN_90	用户按键3	
6	S4	PIN_91	用户按键4	
7	RESET	PIN_24	复位按键	
8	LD1	PIN_28	LED指示灯1	
9	LD2	PIN_30	LED指示灯2	
10	LD3	PIN_31	LED指示灯3	
11	LD4	PIN_32	LED指示灯4	
12	LD5	PIN_33	LED指示灯5	
13	LD6	PIN_34	LED指示灯6	
14	LD7	PIN_38	LED指示灯7	
15	LD8	PIN_39	LED指示灯8	

Figure 5

22	SEGA	PIN_52	数码管A段
23	SEGB	PIN_53	数码管B段
24	SEGC	PIN_54	数码管C段
25	SEGD	PIN_55	数码管D段
26	SEGE	PIN_58	数码管E段
27	SEGF	PIN_59	数码管F段
28	SEGG	PIN_60	数码管G段
29	SEGDP	PIN_64	数码管DP段
30	COM1	PIN_65	数码管位选信号1
31	COM2	PIN_66	数码管位选信号2
32	COM3	PIN_67	数码管位选信号3
33	COM4	PIN_68	数码管位选信号4
34	COM5	PIN_69	数码管位选信号5
35	COM6	PIN_70	数码管位选信号6
36	COM7	PIN_71	数码管位选信号7
37	COM8	PIN_72	数码管位选信号8

Figure 6